Name:khan Ashhad

Roll no:48

subject :ASD&D

Batch:T12

Experiment-02

LO4: To Containerize an Application Using Docker

Aim

To study Docker, understand Docker Images and Containers, and perform basic Docker operations to containerize an application.

Theory

What is Docker?

Docker is an open-source containerization platform that enables developers to build, package, distribute, and run applications in isolated environments called **containers**.

A Docker container packages the application along with all its required dependencies, libraries, runtime, and configuration files. This ensures that the application behaves consistently across different systems regardless of the operating system or installed software.

Docker is lightweight because containers share the host operating system kernel instead of running a complete operating system like a virtual machine.

Need for Docker

In software development, applications often fail when moved from one computer to another because of differences in software versions, operating systems, or missing dependencies.

Docker solves these problems by:

- Packaging the application with all required dependencies.
- Providing the same execution environment on every system.
- Simplifying application deployment.
- Reducing compatibility issues.
- Improving portability and scalability.
- Supporting continuous integration and continuous deployment (CI/CD).

Docker Architecture

Docker architecture mainly consists of the following components.

1. Docker Engine

Docker Engine is the core software responsible for creating, managing, and running Docker containers.

Its functions include:

- Building Docker images
- Running containers
- Managing storage
- Managing Docker networks
- Pulling images from Docker Hub

---

2. Docker Image

A Docker Image is a read-only template used to create Docker containers.

It contains:

- Base operating system
- Application source code
- Runtime environment
- Libraries
- Dependencies
- Configuration files

An image cannot execute directly. It must be used to create a container.

---

3. Docker Container

A Docker Container is a running instance of a Docker image.

It provides an isolated environment in which the application executes.

Characteristics of a container:

- Lightweight
- Portable
- Fast startup
- Isolated from other applications
- Uses fewer system resources than a virtual machine

---

4. Docker Hub

Docker Hub is a cloud-based repository used to store and share Docker images.

Common images available on Docker Hub include:

- Ubuntu
- Python
- Node.js
- MySQL
- PostgreSQL
- MongoDB
- Redis
- Nginx

Docker downloads required images from Docker Hub using the **docker pull** command.

Difference Between Docker Image and Docker Container

Docker Image	Docker Container
Read-only template	Running instance of an image
Used to create containers	Executes the application
Cannot be modified while running	Can be started, stopped and removed
Stored on the system	Runs in memory

Working of Docker

The working of Docker involves the following steps:

1. Develop the application.
2. Create a Dockerfile containing build instructions.
3. Build a Docker Image using the Dockerfile.
4. Store the image locally or on Docker Hub.
5. Create a Docker Container from the image.
6. Execute the application inside the container.

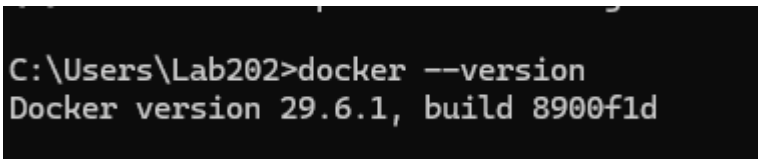
Docker Commands

Command 1: Check Docker Version

Purpose: Displays the installed Docker version.

docker --version

Output Screenshot



Command 2: Display Docker Information

Purpose: Displays detailed information about Docker Engine.

docker info

Output Screenshot

```
C:\Users\Lab202>docker info
Client:
 Version:    29.6.1
 Context:    desktop-linux
 Debug Mode: false
 Plugins:
  agent: Docker AI Agent Runner (Docker Inc.)
    Version:  v1.98.0
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-agent.exe
  ai: Docker AI Agent - Ask Gordon (Docker Inc.)
    Version:  v1.27.0
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-ai.exe
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.35.0-desktop.2
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v5.3.0
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container (Docker Inc.)
    Version:  0.0.47
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-debug.exe
  desktop: Docker Desktop commands (Docker Inc.)
    Version:  v0.4.1
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-desktop.exe
  dhi: CLI for managing Docker Hardened Images (Docker Inc.)
    Version:  v0.0.5
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-dhi.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.31
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-extension.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.4.0
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-init.exe
  mcp: Docker MCP Plugin (Docker Inc.)
    Version:  v0.43.1
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-mcp.exe
  model: Docker Model Runner (Docker Inc.)
    Version:  v1.2.5
    Path:     C:\Users\Lab202\.docker\cli-plugins\docker-model.exe
  offload: Docker Offload (Docker Inc.)
    Version:  v0.6.7
```

```
Images: 2
Server Version: 29.6.1
Storage Driver: overlayfs
 driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
 Volume: local
 Network: bridge host ipvlan macvlan null overlay
 Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
 /etc/cdi
 /var/run/cdi
Swarm: inactive
Runtimes: io.containerd.runc.v2 nvidia runc
Default Runtime: runc
Init Binary: docker-init
containerd version: e53c7c1516c3b2bff98eb76f1f4117477e6f4e66
runc version: v1.3.6-0-g491b69ba
init version: de40ad0
Security Options:
 seccomp
  Profile: builtin
 cgroupns
Kernel Version: 6.18.33.2-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 12
Total Memory: 7.616GiB
Name: docker-desktop
ID: 88912aa9-64f3-4063-81af-798ea9d216d7
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
 com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
```

Command 3: List Available Images

Purpose: Displays all Docker images stored on the system.

docker images

Output Screenshot

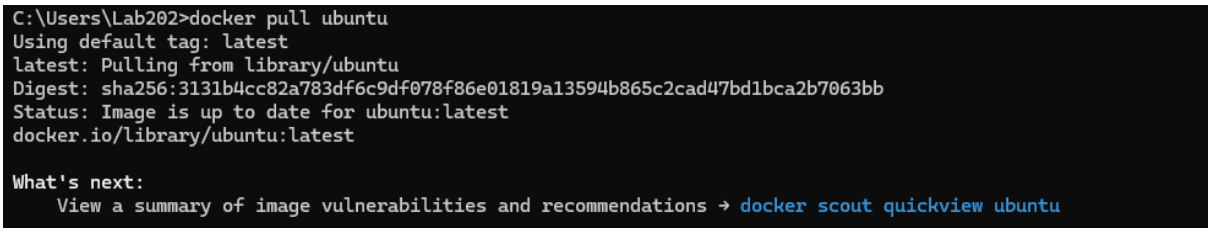


Command 4: Download Ubuntu Image

Purpose: Downloads the Ubuntu image from Docker Hub.

docker pull ubuntu

Output Screenshot



Command 5: Verify Downloaded Image

Purpose: Displays all downloaded Docker images.

docker images

Output Screenshot



Command 6: Run Ubuntu Container

Purpose: Creates and starts a new Ubuntu container.

docker run ubuntu

Output Screenshot

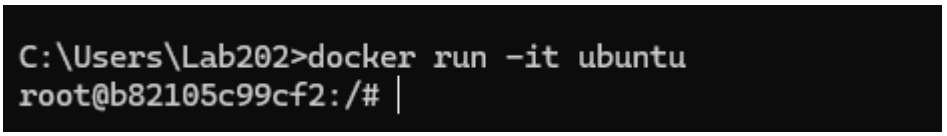


Command 7: Run Ubuntu in Interactive Mode

Purpose: Opens an interactive terminal inside the Ubuntu container.

docker run -it ubuntu

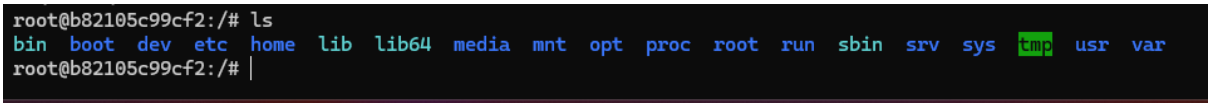
Output Screenshot



Execute the following command inside the container:

ls

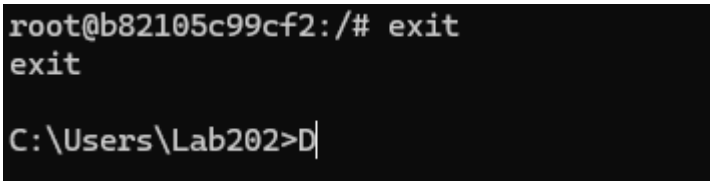
Output Screenshot



Exit the container:

exit

Output Screenshot

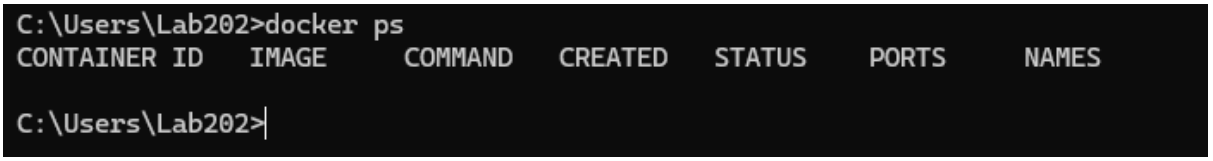


Command 8: Display Running Containers

Purpose: Displays all currently running containers.

docker ps

Output Screenshot

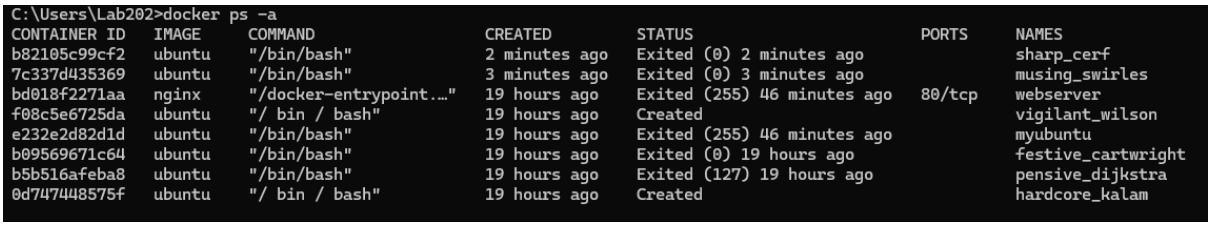


Command 9: Display All Containers

Purpose: Displays all running and stopped containers.

docker ps -a

Output Screenshot

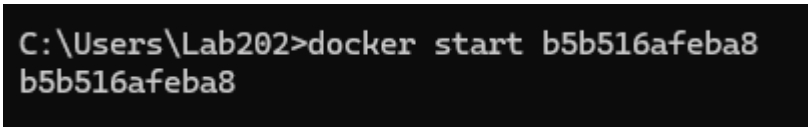


Command 10: Start a Container

Purpose: Starts a stopped container.

docker start <container_id>

Output Screenshot

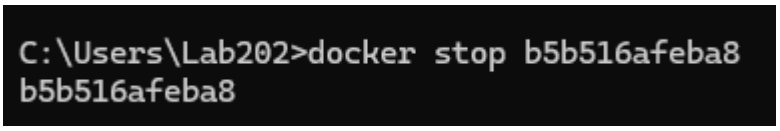


Command 11: Stop a Container

Purpose: Stops a running container.

docker stop <container_id>

Output Screenshot

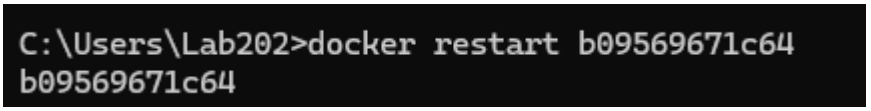


Command 12: Restart a Container

Purpose: Restarts an existing container.

docker restart <container_id>

Output Screenshot

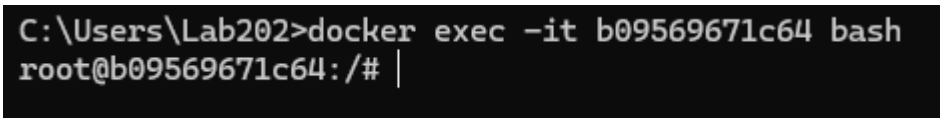


Command 13: Execute Commands Inside a Running Container

Purpose: Opens a terminal inside a running container.

docker exec -it <container_id> bash

Output Screenshot

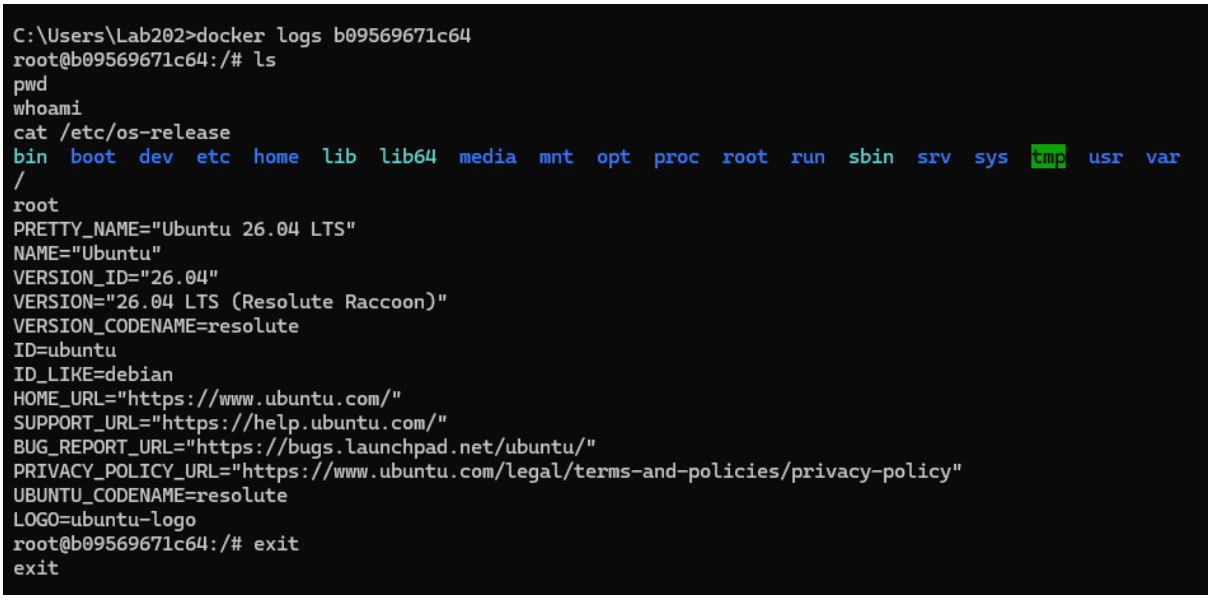


Command 14: View Container Logs

Purpose: Displays log information generated by a container.

docker logs <container_id>

Output Screenshot



Command 15: Remove Unused Docker Resources

Purpose: Removes unused containers, images, cache, and networks.

docker system prune

Output Screenshot

```
C:\Users\Lab202>docker system prune
WARNING! This will remove:
  - all stopped containers
  - all networks not used by at least one container
  - all dangling images
  - unused build cache

Are you sure you want to continue? [y/N] y
Deleted Containers:
b82105c99cf2dc0a9494bdd1fe542ffc16e5a9261cabd0743ea37a0c36a5a9e6
7c337d4353696f304fb42fb3518f076a7618dd032d893abd514b6af9985a786e
bd018f2271aaa7825d3b8cdad78deda49b412fa1ff293b9119c666802bf3e2c9
f08c5e6725da86f22de04d4b01691e6f00eab347a0e32e5eaaccc714cc56da02
e232e2d82d1dbfa79a84954a2e705dbd0717af439365e5c2f31963fb4a265a88
b5b516afeba86fb292a1697f7e2148194dea38ba8e784dd583bfce5179a38d5c
0d747448575fe689f640a9a92620000e6fe547583fbe26fe2286583cffee83c3
```

Advantages of Docker

- Lightweight and efficient.
- Provides isolated execution environments.
- Eliminates dependency conflicts.
- Ensures application portability.
- Simplifies deployment.
- Supports CI/CD pipelines.
- Enables rapid application development.
- Optimizes resource utilization.

Applications of Docker

- Web application deployment
- Microservices architecture
- Cloud computing
- Artificial Intelligence and Machine Learning
- Database deployment
- Software testing
- Continuous Integration and Continuous Deployment (CI/CD)
- DevOps automation

Conclusion:

Successfully studied Docker architecture, understood Docker Images and Docker Containers, and performed Docker commands to create, manage, and execute containers. Thus, the objective of **containerizing an application using Docker** was successfully achieved.